

Dokumentasi Lengkap — Pipeline ETL & ELT

Complete Documentation — ETL & ELT Pipeline

Tugas Besar Big Data · Topik 7: Analitik Keuangan & Transaksi Digital

Credit-Card Default Risk (Taiwan, 2005) + FRED Macro

ID: Dokumen ini menjelaskan **setiap komponen** repo secara detail, lengkap dengan **bukti kode** dan **hasil eksekusi nyata**. Proses langkah-demi-langkah ada di `PROSES_ETL_ELT.pdf`; cara menjalankan ada di `CARA_MENJALANKAN.pdf`.

EN: This document explains **every component** of the repo in detail, with **code evidence** and **real execution outputs**. The step-by-step process is in `PROSES_ETL_ELT.pdf`; how to run is in `CARA_MENJALANKAN.pdf`.

1. Ringkasan & Hasil Nyata / Overview & Real Results

ID: Pipeline mengolah data nasabah kartu kredit (disintesis dengan CTGAN menjadi **150.000** nasabah) digabung dengan data makro FRED Taiwan 2005, melalui **dua jalur**: **ETL** (transformasi di luar warehouse pakai PySpark) dan **ELT** (transformasi di dalam warehouse pakai SQL). Keduanya bermuara di Hive dan divisualkan di Tableau (ETL) & Power BI (ELT).

EN: The pipeline processes credit-card client data (synthesised with CTGAN to **150,000** clients) joined with FRED Taiwan 2005 macro data, through **two paths**: **ETL** (transform outside the warehouse with PySpark) and **ELT** (transform inside the warehouse with SQL). Both land in Hive and are visualised in Tableau (ETL) & Power BI (ELT).

Hasil run nyata / Real run outputs (captured this session):

Tahap / Stage	Output
CTGAN	150,000 clients · class balance {non-default 0.7788, default 0.2212} · seed 42
ETL Transform	900,000 baris fakta (150k × 6 bulan) · 150,000 clients · 6/6 validasi lulus
ETL Load	<code>dim_client</code> 150,000 · <code>dim_date</code> 12 · <code>dim_macro</code> 12 · <code>fact_credit_monthly</code> 900,000
ELT	<code>raw_credit</code> 150,000 · <code>raw_macro</code> 48 · <code>elt_fact_monthly</code> 900,000 · <code>elt_client_trends</code> 900,000 · <code>elt_default_by_demographic</code> 17 · <code>elt_default_vs_macro</code> 6
KPI	overall default rate 0.2212 · female 0.2439 > male 0.1991 · married 0.2661 (tertinggi) · university 0.2504 · usia 40–49 0.2451

2. Arsitektur & Tech Stack / Architecture & Tech Stack

ID: Semua layanan berjalan via **Docker Compose**. Diagram alir data ada di `architecture_diagram.pdf` (§5 CLAUDE.md).

EN: All services run via **Docker Compose**. The data-flow diagram is in `architecture_diagram.pdf` (CLAUDE.md §5).

Komponen	Tools	Peran / Role
Extract	Apache Kafka (KRaft)	producer baca sumber → topic → consumer → raw/+data lake/
Transform (ETL)	Apache Spark / PySpark	clean, unpivot, feature, validasi (row-level)
Transform (ELT)	Hive / Spark SQL	window functions + OLAP GROUPING SETS (set-based)
Warehouse	Apache Hive (metastore + HiveServer2, Postgres backend)	bigdata_etl, bigdata_elt
Synthetic	SDV CTGANSynthesizer	150k nasabah sintetis
Dashboard	Tableau (ETL) + Power BI (ELT) via Hive ODBC	KPI identik di dua tools

3. Infrastruktur Docker / Docker Infrastructure

3.1 docker-compose.yml

ID: Mendefinisikan 7 layanan: kafka (KRaft, tanpa Zookeeper), kafka-ui, spark-master, spark-worker, postgres, hive-metastore, hiveserver2. Volume warehouse dipakai bersama Spark↔Hive pada path **sama** (/opt/hive/data/warehouse) agar path tabel di metastore valid di container Spark.

EN: Defines 7 services. The warehouse volume is shared Spark↔Hive at the **same** path so the absolute table paths stored in the metastore resolve inside the Spark containers too.

Konfigurasi penting pada spark-master (hasil debugging nyata / from real debugging):

```
environment: APP_SPARK_MASTER: "local[*]" HIVE_METASTORE_URI: "thrift://hive-metastore:9083"
KAFKA_BOOTSTRAP_SERVERS: "kafka:19092" USER: "sparkuser" # uid 1001 tak punya entri /etc/passwd → torch
getpass crash HOME: "/tmp" # agar JVM user.home valid (ivy Spark)
```

3.2 docker/Dockerfile.spark

ID: Image Spark kustom. **Catatan penting hasil run:** image dasar bitnami lama (bitnami/spark) sudah dihapus 2025 → memakai bitnamilegacy/spark:3.5. pyspark **tidak** di-pip-install (sudah ada di base image, sdist 317 MB). Ditambahkan entri /etc/passwd untuk uid 1001 agar getpwuid() & user.home JVM tidak error.

EN: Custom Spark image. **Key run findings:** the old bitnami/spark was removed in 2025 → use bitnamilegacy/spark:3.5; pyspark is **not** re-installed (already in base); a /etc/passwd entry for uid 1001 fixes getpwuid()/JVM user.home.

```
FROM bitnamilegacy/spark:3.5 USER root COPY requirements.txt /tmp/requirements.txt RUN grep -ivE
'^pyspark([=<>!~ ]|$)' /tmp/requirements.txt > /tmp/req.spark.txt \ && pip install --no-cache-dir -r
/tmp/req.spark.txt RUN echo 'sparkuser:x:1001:0:sparkuser:/tmp:/bin/bash' >> /etc/passwd USER 1001
```

3.3 docker/Dockerfile.hive

ID: Image Hive 4.0 + driver JDBC PostgreSQL. **Catatan:** ADD <url> menghasilkan file 0 byte (redirect Maven) → diganti COPY jar yang di-download dulu di host.

EN: Hive 4.0 image + PostgreSQL JDBC driver. **Note:** ADD <url> produced a 0-byte file (Maven redirect) → replaced with COPY of a host-downloaded jar.

```
FROM apache/hive:4.0.0 USER root COPY docker/postgresql-42.7.4.jar /opt/hive/lib/postgresql-42.7.4.jar RUN
chmod 644 /opt/hive/lib/postgresql-42.7.4.jar USER hive
```

4. Paket Bersama / Shared `common/` Package

ID: Utilitas dipakai semua tahap: `paths.py` (direktori), `logging_utils.py` (logger konsisten), `metrics.py` (instrumentasi runtime/memori untuk perbandingan \$9), `spark_session.py` (SparkSession + Hive).

EN: Shared utilities across all stages.

`common/spark_session.py` — konfigurasi kunci hasil debugging / key configs from debugging:

```
builder = ( SparkSession.builder.appName(app_name).master(master) .config("spark.sql.warehouse.dir",
HIVE_WAREHOUSE) .config("spark.sql.hive.metastore.version", "2.3.9") .config("spark.sql.hive.metastore.jars",
"builtin") .config("spark.jars.ivy", "/tmp/.ivy2") # ivy basedir absolut
.config("spark.hadoop.fs.permissions.umask-mode", "000") # warehouse ditulis 2 uid ) if with_hive: builder =
builder.config("hive.metastore.uris", metastore_uri).enableHiveSupport()
```

`common/metrics.py` — context manager pencatat metrik per tahap / per-stage metric recorder:

```
@contextmanager def stage_timer(pipeline, stage, logger=None, **extra): metric =
StageMetric(pipeline=pipeline, stage=stage, extra=dict(extra)) start = time.perf_counter() try: yield metric
finally: metric.duration_s = round(time.perf_counter() - start, 3) record(metric) # ->
logs/pipeline_metrics.jsonl
```

5. ETL — Extract (`etl_pipeline/extract.py`)

ID: Menarik 2 sumber ke `raw/` **tanpa cleaning**, mencatat log terstruktur (nama, baris, kolom, ukuran, durasi, status).

EN: Pulls 2 sources to `raw/` **without cleaning**, logging structured metadata.

- `extract_etl_source1()` — UCI *Default of Credit Card Clients* (id 350), file `.xls` mentah.
- `extract_etl_source2()` — 4 seri FRED (EXTAUS, RBTWBIS, NBTWBIS, TRESEGTWM194N), 2005, JSON.

```
def extract_etl_source2(raw_dir=RAW_DIR, api_key=None, year=DEFAULT_YEAR, series_ids=None): api_key = api_key
or os.getenv("FRED_API_KEY") for series_id in (series_ids or FRED_SERIES): params = {"series_id": series_id,
"api_key": api_key, "file_type": "json", "observation_start": f"{year}-01-01", "observation_end":
f"{year}-12-31"} resp = requests.get(FRED_BASE_URL, params=params, timeout=HTTP_TIMEOUT)
out_path.write_text(json.dumps(payload, indent=2), encoding="utf-8") # raw JSON
```

Hasil / Result: UCI 30,000×25; tiap seri FRED 12 observasi (Jan–Des 2005).

6. ETL — Kafka (`kafka_producer.py`, `kafka_consumer.py`)

ID: Membungkus extract dengan Kafka (\$5 arsitektur): producer publish tiap baris/observasi ke topic `credit-card-raw` & `fred-macro-raw`; consumer menulis ke `raw/` (ETL) **dan** `datalake/` (ELT). Uji cepat: `--limit N`.

EN: Wraps extract with Kafka; consumer writes to both `raw/` and `datalake/`. Smoke test: `--limit N`.

```
producer.send(topic, key=rec.get(key_field), value=rec) # producer # consumer: for msg in consumer: (credit if
msg.topic == TOPIC_CREDIT else fred).append(msg.value) _write_both("credit_card_stream.csv", lambda p:
df.to_csv(p, index=False)) # raw/ + datalake/
```

7. Synthetic — CTGAN ([synthetic/train_ctgan.py](#))

ID: Melatih CTGAN (SDV) pada data UCI, generate $\geq 150,000$ nasabah, **menjaga keseimbangan kelas** target via *conditional sampling*, seed tetap. **Catatan run:** dijadikan **pra-langkah sekali jalan** (berat/torch) lalu output disalin ke `raw/` sebagai sumber 1 — supaya tidak bentrok memori dengan Hive (OOM).

EN: Trains CTGAN on UCI data, generates $\geq 150,000$ clients, **preserves target class balance** via conditional sampling, fixed seed. **Run note:** treated as a **one-time pre-step** (heavy/torch); its output is copied to `raw/` as source 1 to avoid OOM contention with Hive.

```
synth = CTGANSynthesizer(metadata, epochs=args.epochs, verbose=True) synth.fit(real_features) # ID dibuang
konsisten dgn metadata conditions = [Condition(num_rows=int(n), column_values={target: c}) for c, n in
counts.items()] synthetic = synth.sample_from_conditions(conditions=conditions) # balance terkontrol
```

Hasil / Result: 150,000 baris, balance {0: 0.7788, 1: 0.2212} (mempertahankan ~22% asli).

Detail bias/limitasi: lihat `synthetic/CTGAN_METHOD.md`.

8. ETL — Transform ([etl_pipeline/transform.py](#), PySpark)

ID: Transformasi *row-level* (§6.2). Sumber kredit = `raw/synthetic_credit_clients.csv`.

EN: Row-level transform. Credit source = `raw/synthetic_credit_clients.csv`.

a. Cleaning — dedup PK, missing per dtype, outlier **IQR** (kredit) & **Z-score** (makro).

```
def iqr_cap(df, col): q1, q3 = df.approxQuantile(col, [0.25, 0.75], 0.01) iqr = q3 - q1; lo, hi = q1 -
1.5*iqr, q3 + 1.5*iqr return df.withColumn(col, F.when(F.col(col) < lo, lo).when(F.col(col) > hi,
hi).otherwise(F.col(col)))
```

b. Standardisasi — `snake_case`; normalisasi min-max `exchange_rate_twd_usd`, `real_broad_eer`, `total_reserves`; encode kategorikal (label `sex/education/marriage`).

c. Enrichment — **unpivot** wide→bulanan (Apr–Sep 2005) via union 6 bagian; join makro lewat `dim_date`; **5 fitur:** `credit_utilization`, `payment_ratio`, `avg_delay_months`, `num_months_delayed`, `repayment_gap`.

```
MONTH_MAP = [(200509,9,"pay_0","bill_amt1","pay_amt1"), (200508,8,"pay_2","bill_amt2","pay_amt2"), ...] fact =
fact.withColumn("credit_utilization",
F.round(F.col("bill_amt")/F.when(F.col("limit_bal")==0,None).otherwise(F.col("limit_bal")),4))
```

d. Validasi — **6 aturan / 6 rules:** uniqueness, null, range, dtype, referential integrity, distribution. **Hasil nyata / real:** semua **PASS**; `default_rate=0.2212`; `neg_utilization_kept` dilaporkan (utilisasi negatif = akun overpaid, **valid**).

Output: `parquet staging warehouse/staging/{fact_credit_monthly,dim_client,dim_date,dim_macro}`.

9. ETL — Load + Star Schema ([etl_pipeline/load.py](#), `warehouse/etl_*.sql`)

ID: Membuat **star schema** Hive dari staging, menerapkan PK/FK (informational), menjalankan

8 query analitik. DB di-DROP ... CASCADE lalu dibuat ulang → run **idempoten**.

EN: Builds the Hive **star schema** from staging, applies PK/FK, runs **8 analytical queries**.

DB is dropped+recreated → idempotent runs.

```
spark.sql(f"DROP DATABASE IF EXISTS {DB} CASCADE"); spark.sql(f"CREATE DATABASE {DB}")
df.write.mode("overwrite").format("parquet").saveAsTable(f"{DB}.{tbl}") spark.sql("ALTER TABLE
bigdata_etl.fact_credit_monthly ADD CONSTRAINT fk_fact_date " "FOREIGN KEY (date_key) REFERENCES
bigdata_etl.dim_date(date_key) DISABLE NOVALIDATE RELY")
```

Skema (grain = client × bulan) / star schema: fact_credit_monthly → dim_client, dim_date;
dim_macro via dim_date.

Hasil 8 query (nyata) / 8-query results (real):

- Q1 overall default rate = **0.2212** (150,000).
- Q2 sex: female 0.2439 · male 0.1991.
- Q3 education: university 0.2504 · high_school 0.241 · graduate_school 0.2117.
- Q4 marriage: married 0.2661 · single 0.1992.
- Q5 age_band: 40–49 0.2451 (tertinggi) · 60+ 0.1568 (terendah).
- Q6 tren bulanan: utilisasi naik 0.306→0.478 (Apr→Sep).
- Q7 default vs makro per bulan (FX 31.48→32.92, reserves naik).
- Q8 korelasi default↔makro = **0.0** — *artifact*: label default bersifat client-level (sama di 6 bulan) sehingga variansi bulanannya nol → korelasi dengan makro nol (didokumentasikan jujur).

10. ELT — Pipeline (elt_pipeline/)

ID: Memuat sumber **mentah** (tanpa preprocessing) ke tabel Hive raw, lalu transformasi **di dalam warehouse** dengan SQL set-based — pendekatan **berbeda** dari ETL.

EN: Loads **raw** sources (no preprocessing) into Hive raw tables, then transforms **inside the warehouse** with set-based SQL — a genuinely **different** approach from ETL.

extract_load.py → bigdata_elt.raw_credit (150,000×25), raw_macro (48). transform.sql (CTAS
USING PARQUET):

```
-- unpivot di SQL (bukan Python) via stack() CREATE TABLE bigdata_elt.elt_fact_monthly USING PARQUET AS SELECT
c.id, m.date_key, c.limit_bal, m.pay_status, m.bill_amt, m.pay_amt, ... FROM bigdata_elt.raw_credit c LATERAL
VIEW stack(6, 200509,c.pay_0,c.bill_amt1,c.pay_amt1, 200508,c.pay_2,c.bill_amt2,c.pay_amt2, ...) m AS ...; --
WINDOW functions: tren month-over-month + ranking LAG(bill_amt) OVER (PARTITION BY id ORDER BY date_key) AS
prev_bill_amt, RANK() OVER (PARTITION BY date_key ORDER BY bill_amt DESC) AS bill_rank_in_month -- OLAP:
GROUPING SETS (per dimensi + grand total) GROUP BY sex_label, education_label, marriage_label, age_band
GROUPING SETS ((sex_label),(education_label),(marriage_label),(age_band),());
```

Hasil / Result: elt_fact_monthly 900,000 · elt_client_trends 900,000 · elt_default_by_demographic
17 baris (2 sex + 5 edu + 4 marriage + 5 age + 1 total) · elt_default_vs_macro 6.

11. Dashboard Layer (warehouse/dashboard_kpis.sql, dashboard/)

ID: Tabel KPI kpi_* dibuat **identik** di kedua DB (bigdata_etl→Tableau, bigdata_elt→Power BI), materialised USING PARQUET agar terbaca Hive ODBC. Spesifikasi visual: DASHBOARD_SPEC.md; koneksi ODBC: HIVE_ODBC_CONNECTION.md.

EN: kpi_* tables are identical in both DBs, materialised USING PARQUET for Hive ODBC.

KPI: kpi_overall, kpi_default_by_demographic, kpi_monthly_default_vs_macro, kpi_corr_default_macro.

Terverifikasi via beeline / verified via beeline: kpi_overall = 0.2212 / 150,000.

12. Analisis ETL vs ELT (analysis/)

ID: compare_pipelines.py membaca logs/pipeline_metrics.jsonl (dari stage_timer) → etl_vs_elt_metrics.{csv,md}. Diskusi kualitatif (runtime/tahap, kompleksitas, posisi beban transform, freshness) di ETL_vs_ELt_comparison.md.

EN: compare_pipelines.py reads the metrics JSONL → comparison table; qualitative discussion in ETL_vs_ELt_comparison.md. **Inti / core:** ETL = beban transform di hulu (Python/Spark); ELT = beban di hilir, di dalam warehouse (SQL window/OLAP), data mentah cepat tersedia.

13. Kepatuhan Spesifikasi / Spec Compliance (brief §4)

- ■ $\geq 100,000$ baris $\rightarrow$ **900,000** baris fakta ($150k \times 6$ bulan).
- ■ ≥ 12 kolom $\rightarrow$ 24 kredit + ≥ 5 fitur (+ makro).
- ■ ≥ 2 sumber (UCI + FRED) · ■ ≥ 2 format (CSV/XLSX + JSON).
- ■ numerik + kategorikal + datetime · ■ missing & duplikat ditangani · ■ PK/FK (`id`, `date_key`).
- ■ metode sintetik terdokumentasi (`CTGAN_METHOD.md`).